Department of
Computer Science and Engineering

Title: Resource Allocation Graph Deadlock detection

Operating System Lab

CSE 310

Green University of Bangladesh

1 Objective(s)

- To gather knowledge of Deadlock Detection using Resource Allocation Graph (RAG).
- To implement Deadlock detection using Cycle detection method.

2 Problem analysis

A deadlock is a situation in which two computer programs are sharing the same resource and effectively preventing each other from accessing the resource, resulting in both programs ceasing to function. The following conditions are necessary and sufficient for a system to be in deadlock:

1. Mutual exclusion: multiple threads cannot simultaneously hold the same resource.
2. Hold and wait: threads can hold one resource while waiting for another.
3. No preemption: it is impossible to force a thread to relinquish a resource until it has completed its task.
4. Circular wait: there is a thread that is currently waiting for another thread, which is currently waiting for another thread, ..., which is currently waiting on the first thread.

2.1 Deadlock Detection using Resource Allocation Graph (RAG)

A resource allocation graphs shows which resource is held by which process and which process is waiting for a resource of a specific kind. It is amazing and straight-forward tool to outline how interacting processes can deadlock. Therefore, resource allocation graph describe what the condition of the system as far as process and resources are concern like what number of resources are allocated and what is the request of each process. Everything can be represented in terms of graph. The circular wait condition can be easily explained using a resource allocation graph. The graph is drawn according to the following rules:

1. Vertices represent either threads (drawn as ovals) or resources (drawn as rectangles).
2. The directed edge from a process P to a resource R represents that the process P is willing to get resource R
3. The directed edge from a resource R to a process P represents that the process P is holding the resource R in its custody.

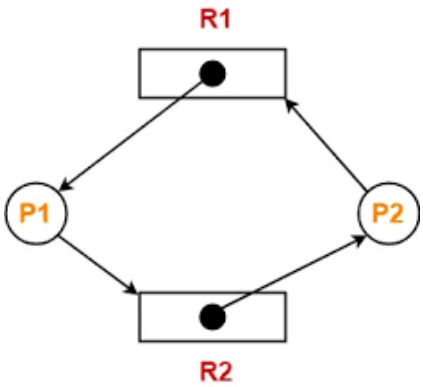

Figure 1: A simple RAG

If there is a cycle in the Resource Allocation Graph and each resource in the cycle provides only one instance, then the processes will be in deadlock. For example, if process P1 holds resource R1, process P2 holds resource R2 and process P1 is waiting for R2 and process P2 is waiting for R1, then process P1 and process P2 will be in deadlock.

© Dept. of Computer Science and Engineering, GUB

3 Deadlock detection using Cycle detection method

If a cycle is being formed in a Resource allocation graph where all the resources have the single instance then the system is deadlocked.
In Case of Resource allocation graph with multi-instanced resource types, Cycle is a necessary condition of deadlock but not the sufficient condition.

4 Implementation in C

```c
#include <stdio.h>
static int mark[20];
int i, j, np, nr;

int main() {
  int alloc[10][10], request[10][10], avail[10], r[10], w[10];

  printf("\nEnter the no of process: ");
  scanf("%d", &np);
  printf("\nEnter the no of resources: ");
  scanf("%d", &nr);
  for (i = 0; i < nr; i++) {
    printf("\nTotal Amount of the Resource R%d: ", i + 1);
    scanf("%d", &r[i]);
  }

  printf("\nEnter the request matrix:");

  for (i = 0; i < np; i++)
    for (j = 0; j < nr; j++) scanf("%d", &request[i][j]);

  printf("\nEnter the allocation matrix:");
  for (i = 0; i < np; i++)
    for (j = 0; j < nr; j++) scanf("%d", &alloc[i][j]);
  /*Available Resource calculation*/
  for (j = 0; j < nr; j++) {
    avail[j] = r[j];
    for (i = 0; i < np; i++) {
      avail[j] -= alloc[i][j];
    }
  }

  // marking processes with zero allocation

  for (i = 0; i < np; i++) {
    int count = 0;
    for (j = 0; j < nr; j++) {
      if (alloc[i][j] == 0)
        count++;
      else
        break;
    }
    if (count == nr) mark[i] = 1;
  }
  // initialize W with avail

  for (j = 0; j < nr; j++) w[j] = avail[j];

```

```
  // mark processes with request less than or equal to W
  for (i = 0; i < np; i++) {
    int canbeprocessed = 0;
    if (mark[i] != 1) {
      for (j = 0; j < nr; j++) {
        if (request[i][j] <= w[j])
          canbeprocessed = 1;
        else {
          canbeprocessed = 0;
          break;
        }
      }
      if (canbeprocessed) {
        mark[i] = 1;

        for (j = 0; j < nr; j++) w[j] += alloc[i][j];
      }
    }
  }

  // checking for unmarked processes
  int deadlock = 0;
  for (i = 0; i < np; i++)
    if (mark[i] != 1) deadlock = 1;

  if (deadlock)
    printf("\n Deadlock detected");
  else
    printf("\n No Deadlock possible");
}
```

5 Input/Output

5.1 Input

Input of the program is given below.

```
Enter the no of process: 4
Enter the no of resources: 5

Total Amount of the Resource R1: 2
Total Amount of the Resource R2: 1
Total Amount of the Resource R3: 1
Total Amount of the Resource R4: 2
Total Amount of the Resource R5: 1

Enter the request matrix:0 1 0 0 1
0 0 1 0 1
0 0 0 0 1
1 0 1 0 1

Enter the allocation matrix:1 0 1 1 0
1 1 0 0 0
0 0 0 1 0
0 0 0 0 0
```

© Dept. of Computer Science and Engineering, GUB

5.2 Output

Output of the program is given below.

```
Deadlock detected
```

6 Discussion & Conclusion

Based on the focused objective(s) to understand about deadlock detection algorithm using Resource Allocation graph, the additional lab exercise made more confident towards the fulfillment of the objectives(s).

7 Lab Task (Please implement yourself and show the output to the instructor)

1. Implement deadlock detection using RAG which detects the deadlock nodes.

7.1 Problem analysis

The objective is to implement a deadlock detection algorithm using a Resource Allocation Graph (RAG). Deadlocks occur in systems where multiple processes compete for resources, and they can lead to system-wide halts if not managed properly. Detecting deadlocks allows the system to take corrective actions to resolve them. The input consists of nodes representing processes and resources, and edges representing resource allocations. Each edge connects a resource to a process indicating that the process is holding that resource. The output is the identification of deadlock nodes if a deadlock is detected in the system. A deadlock node is a node in the graph where a deadlock occurs.

7.2 Input

Input of the program is given below.

```
Enter the no of process: 4
Enter the no of resources: 5

Total Amount of the Resource R1: 2
Total Amount of the Resource R2: 1
Total Amount of the Resource R3: 1
Total Amount of the Resource R4: 2
Total Amount of the Resource R5: 1

Enter the request matrix:0 1 0 0 1
0 0 1 0 1
0 0 0 0 1
1 0 1 0 1

Enter the allocation matrix:1 0 1 1 0
1 1 0 0 0
0 0 0 1 0
0 0 0 0 0
```

7.3 Output

Output of the program is given below.

```
Deadlock detected on the Nodes 1,2,3,4
```

8 Lab Exercise (Submit as a report)

- Implement deadlock avoidance algorithm (Banker's Algorithm).

8.1 Input

Input of the program is given below.

```
Enter the number of resources : 3
Enter the max instances of each resource
a= 10
b= 5
c= 7
Enter the number of processes: 5
Enter the allocation matrix
a b c
P[0] 0 1 0
P[1] 2 0 0
P[2] 3 0 2
P[3] 2 1 1
P[4] 0 0 2

Enter the MAX matrix
a b c
P[0] 7 5 3
P[1] 3 2 2
P[2] 9 0 2
P[3] 4 2 2
P[4] 5 3 3
```

8.2 Output

Output of the program is given below.

```
< P[1] P[3] P[4] P[0] P[2] >
```

9 Policy

Copying from internet, classmate, seniors, or from any other source is strongly prohibited. 100% marks will be *deducted* if any such copying is detected.

© Dept. of Computer Science and Engineering, GUB